

Yep — these are exactly the little details that make the whole Proxy/routing picture click.
Let's go carefully. 😊

1. What exactly is 308 ?

308 is an HTTP status code meaning:

Permanent Redirect

Next.js uses **308 by default** when you ask for a permanent redirect.  Next.js

For example:

```
permanentRedirect("/new-url");
```

results conceptually in:

```
HTTP/1.1 308 Permanent Redirect
Location: /new-url
```



Why 308 instead of 301?

The important distinction is **HTTP method preservation**.

Suppose:

```
POST /old-api
```



gets redirected.

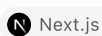
With a 308 :

```
POST /old-api
  ↓
308
  ↓
POST /new-api
```



The **POST** method is preserved.

A **301** can allow a client to change the request from **POST** to **GET** ; **308** doesn't.



So:

Code	Meaning	Method preserved?
301	Permanent redirect	Not guaranteed

Code	Meaning	Method preserved?
302	Temporary redirect	Not guaranteed
307	Temporary redirect	✓
308	Permanent redirect	✓

For your Next.js interview:

308 is a permanent HTTP redirect that preserves the original HTTP method.
Next.js uses 308 for `permanentRedirect()` and `permanent: true` redirects.

That's enough.

2. Now Proxy: "before normal route handling" — URL or API?

Both. This is the important part.

Proxy operates on the **incoming request**, before Next.js proceeds with the normal handling of that request.

That request could be:

GET /dashboard



or:

GET /products/123



or:

POST /api/orders



etc.

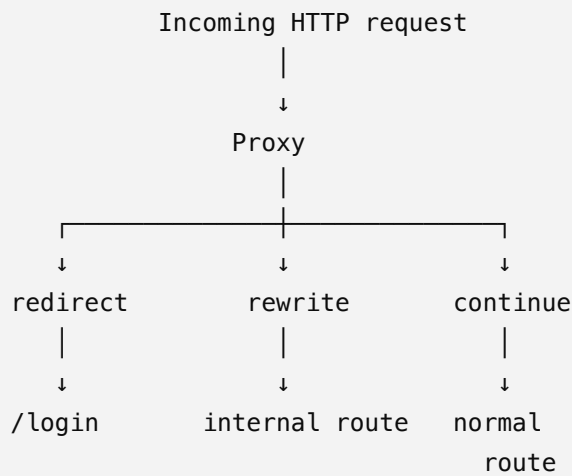
So don't think:

"Proxy happens before API."

Think:

"Proxy can intercept matching incoming requests before they continue to their normal destination."

Conceptually:



For a page:

```
GET /dashboard
  ↓
Proxy
  ↓
dashboard/page.tsx
```

For an API endpoint:

```
GET /api/products
  ↓
Proxy
  ↓
/api/products/route.ts
```

provided your matcher includes that API path.

3. What exactly is **matcher** ?

This is VERY important.

Your Proxy does **not necessarily** run for every request.

You tell Next.js which paths should invoke it using:

```
export const config = {  
  matcher: [...]  
};
```

Think of `matcher` as:

"Which incoming URLs should enter my Proxy?"

Simple example

Suppose:

```
app/  
├─ page.tsx  
├─ dashboard/  
│   ├─ page.tsx  
│   └─ settings/  
│       └─ page.tsx  
├─ products/  
│   └─ page.tsx  
└─ proxy.ts
```



You want authentication only for dashboard routes.

`proxy.ts`

```
import { NextRequest, NextResponse } from "next/server";  
  
export function proxy(request: NextRequest) {  
  const token = request.cookies.get("token");  
  
  if (!token) {  
    return NextResponse.redirect(  
      new URL("/login", request.url)  
    );  
  }  
  
  return NextResponse.next();  
}  
  
export const config = {  
  matcher: ["/dashboard/:path*"],  
};
```

Now look at these requests:

/dashboard



matches:

/dashboard/:path*



✓ Proxy runs.

/dashboard/settings



matches.

✓ Proxy runs.

/dashboard/orders/123



matches.

✓ Proxy runs.

/products



doesn't match.

✗ Proxy doesn't run.

/about



doesn't match.

✗ Proxy doesn't run.

What does `:path*` mean?

This:

```
/dashboard/:path*
```



means:

```
/dashboard  
/dashboard/anything  
/dashboard/anything/else  
/dashboard/a/b/c
```



So `:path*` means:

zero or more path segments after `/dashboard`.

That's why it catches the entire dashboard subtree.

4. Multiple matchers

You can have:

```
export const config = {  
  matcher: [  
    "/dashboard/:path*",  
    "/admin/:path*",  
  ],  
};
```

Now:

```
/dashboard  
/dashboard/settings  
/admin  
/admin/users
```



go through Proxy.

But:

```
/products  
/about
```



don't.

Think:

matcher



Which requests do I
want Proxy to see?



5. Real authentication example

Let's make the whole flow concrete.

Suppose your application has:

```
/login  
/dashboard  
/dashboard/profile  
/dashboard/settings  
/products  
/api/products
```



You want:

```
/dashboard/*
```



to require login.

proxy.ts

```
import { NextRequest, NextResponse } from "next/server";  
  
export function proxy(request: NextRequest) {  
  const token = request.cookies.get("token");  
  
  if (!token) {  
    return NextResponse.redirect(  
      new URL("/login", request.url)  
    );  
  }  
  
  return NextResponse.next();  
}  
  
export const config = {
```

```
matcher: ["/dashboard/:path*"],  
};
```

Now:

Request 1

GET /products

```
  /products  
    ↓  
  matcher  
    ↓  
 NO MATCH  
    ↓  
 Proxy skipped  
    ↓  
products/page.tsx
```

Request 2

GET /dashboard

```
  /dashboard  
    ↓  
  matcher  
    ↓  
  MATCH  
    ↓  
  Proxy  
    ↓  
token exists?  
  /      \  
YES      NO  
  ↓      ↓  
next()  redirect  
  ↓      ↓  
dashboard  /login
```

That's the complete mechanism.

Next.js itself documents `matcher` as the mechanism for specifying which paths Proxy should run on.  Next.js

6. What about API routes?

You can explicitly match them too.

Suppose:

```
app/api/products/route.ts
```



is your API.

You could write:

```
export const config = {  
  matcher: [  
    "/dashboard/:path*",  
    "/api/:path*",  
  ],  
};
```

Now:

```
/dashboard  
/dashboard/settings  
/api/products  
/api/orders  
/api/users/123
```



all enter Proxy.

Example

```
export function proxy(request: NextRequest) {  
  const token = request.cookies.get("token");  
  
  if (!token) {  
    return NextResponse.json(  
      { error: "Unauthorized" },  
      { status: 401 }  
    );  
  }  
  
  return NextResponse.next();  
}
```

```
export const config = {  
  matcher: ["/api/:path*"],  
};
```

Then:

```
GET /api/products  
  ↓  
Proxy  
  ↓  
token?  
  ↓  
YES  
  ↓  
NextResponse.next()  
  ↓  
api/products/route.ts
```



So yes — **Proxy can sit in front of API routes too**, depending on your matcher.

7. Now let's understand `NextResponse.next()`

This:

```
return NextResponse.next();
```

basically means:

"I have inspected this request; let it continue to the normal destination."

For example:

```
export function proxy(request: NextRequest) {  
  const token = request.cookies.get("token");  
  
  if (!token) {  
    return NextResponse.redirect(  
      new URL("/login", request.url)  
    );  
  }  
}
```

```
return NextResponse.next();  
}
```

There are essentially three interesting outcomes:

Proxy

```
├── redirect()  
│   ↓  
│   send somewhere else  
├── rewrite()  
│   ↓  
│   internally serve another route  
└── next()  
    ↓  
    continue normally
```

That's a fantastic mental model.

8. Now let's do a REAL multi-route rewrite

You asked:

"if it's multi route how does it recognize?"

Let's use a multi-tenant application because this is where rewrites become extremely useful.

Imagine you have:

```
alice.myapp.com  
bob.myapp.com
```

And internally your application has:

```
app/  
├── _sites/  
│   ├── alice/  
│   │   └── page.tsx  
│   └── bob/  
│       └── page.tsx  
└── proxy.ts
```

But you don't want users to see:

/_sites/alice



They should simply see:

alice.myapp.com



Proxy

```
import { NextRequest, NextResponse } from "next/server";

export function proxy(request: NextRequest) {
  const hostname = request.headers.get("host");

  if (hostname === "alice.myapp.com") {
    return NextResponse.rewrite(
      new URL("/_sites/alice", request.url)
    );
  }

  if (hostname === "bob.myapp.com") {
    return NextResponse.rewrite(
      new URL("/_sites/bob", request.url)
    );
  }

  return NextResponse.next();
}
```

Now let's follow a request.

Alice visits

https://alice.myapp.com/



Incoming request:

GET /
Host: alice.myapp.com



Proxy runs.

It reads:

```
const hostname = request.headers.get("host");
```

gets:

```
alice.myapp.com
```



Then:

```
NextResponse.rewrite(  
  new URL("/_sites/alice", request.url)  
)
```

So internally:

```
alice.myapp.com/  
  ↓  
  Proxy  
  ↓  
hostname = alice.myapp.com  
  ↓  
rewrite  
  ↓  
/_sites/alice  
  ↓  
app/_sites/alice/page.tsx
```



But the browser still sees:

```
alice.myapp.com/
```



That's the whole point.

Bob

```
https://bob.myapp.com/
```



Request:

Host: bob.myapp.com



Proxy:

```
bob.myapp.com
  ↓
rewrite
  ↓
/_sites/bob
```



So:

```
alice.myapp.com
  ↓
/_sites/alice

bob.myapp.com
  ↓
/_sites/bob
```



This is how Proxy can use request information to select different internal routes.

9. But how does the matcher work here?

We don't necessarily want to intercept every single possible request.

For example, maybe only website pages should be handled this way.

We could use:

```
export const config = {
  matcher: [
    "/((?!api|_next/static|_next/image|favicon.ico).*)",
  ],
};
```

This says, roughly:

Match application paths while excluding API requests, Next.js static assets, optimized images, and favicon.

The Next.js authentication example uses a similar negative-lookahead matcher to exclude API/static/image paths.  Next.js

But honestly, **don't memorize this regex.**

For interviews, know the concept:

```
matcher
  ↓
select which requests Proxy should process
```



and understand simple patterns:

```
matcher: ["/dashboard/:path*"]
```

or:

```
matcher: ["/api/:path*"]
```

That's much more useful.

10. Why not just redirect for multi-tenancy?

Because redirect would expose the internal route.

Imagine:

```
alice.myapp.com
  ↓
redirect
  ↓
alice.myapp.com/_sites/alice
```



Now the browser URL becomes:

```
alice.myapp.com/_sites/alice
```



You probably don't want that.

With rewrite:

```
alice.myapp.com/
  ↓
rewrite
```



↓
/_sites/alice

browser still shows:

alice.myapp.com/



That's the key reason for rewriting.

11. One more very practical example: API versioning

Suppose your public API is:

/api/products



but internally you migrated to:

/api/v2/products



You don't want all existing clients to suddenly change their URLs.

Proxy:

```
export function proxy(request: NextRequest) {  
  if (request.nextUrl.pathname === "/api/products") {  
    return NextResponse.rewrite(  
      new URL("/api/v2/products", request.url)  
    );  
  }  
  
  return NextResponse.next();  
}
```

Request:

GET /api/products



Flow:

Browser/client
↓
/api/products
↓



```
Proxy
  ↓
rewrite
  ↓
/api/v2/products
  ↓
route handler
```

The client still thinks it called:

```
/api/products
```



but internally you're using:

```
/api/v2/products
```



12. Redirect vs Rewrite — visualize it

Redirect

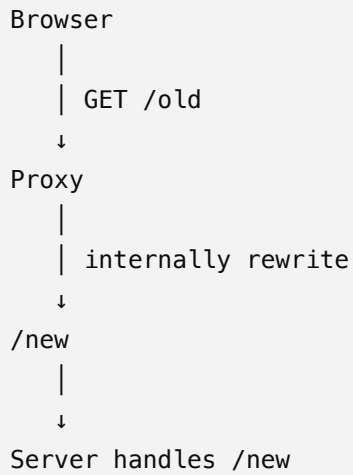
```
Browser
  |
  | GET /old
  ↓
Proxy/Server
  |
  | 308 Location: /new
  ↓
Browser
  |
  | GET /new
  ↓
Server
  |
  ↓
/new
```



Two requests.

Browser URL changes.

Rewrite



```
graph TD; Browser -- GET /old --> Proxy; Proxy -- internally rewrite --> NewPath[/new/]; NewPath --> Server[Server handles /new];
```

Browser

GET /old

Proxy

internally rewrite

/new

Server handles /new

No second browser request.

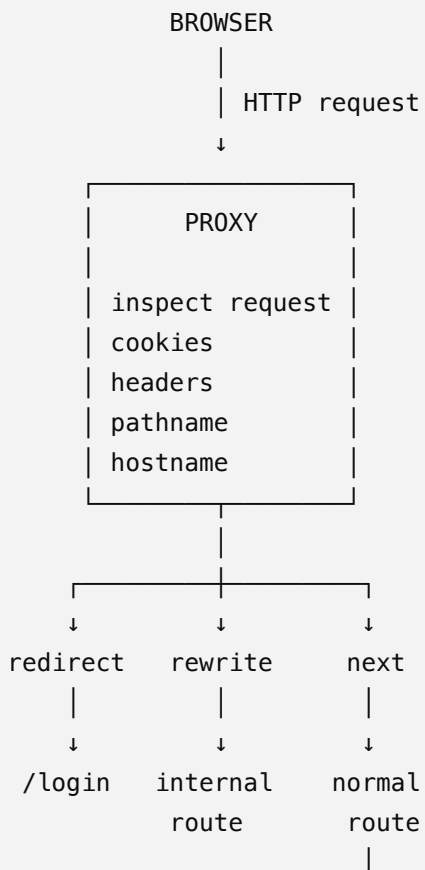
Browser URL stays:

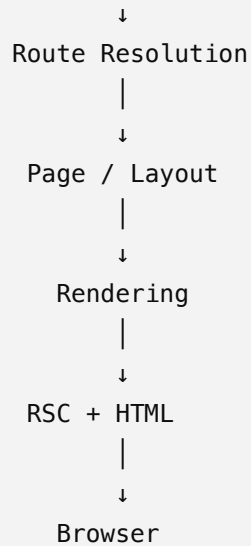
/old

This is the cleanest way to remember it.

13. And where exactly is Proxy in the lifecycle?

For a matching request, think:





And importantly:

"Before normal route handling" doesn't mean Proxy literally runs before the URL exists or before HTTP routing at the network level. The request already has a URL. It means Next.js gives Proxy an opportunity to process the matching request before the application's normal route handling/rendering continues.

That's the precise mental model.

What I'd say in an interview

If they ask "What is Proxy/Middleware in Next.js?", I'd say:

"Proxy is a request interception layer that can run before the normal handling of matching requests. I can use a matcher to specify which paths it applies to. From Proxy, I can inspect things like the pathname, cookies, headers, or hostname and then either continue with `NextResponse.next()`, redirect the request, or rewrite it to another internal route. A common use case is protecting dashboard routes based on authentication, while rewrites are useful for things like multi-tenancy where different hostnames map to different internal routes without changing the browser URL."

That's a **very strong answer** because it covers:

```
what it is
+
when it runs
+
matcher
+
request information
+
```



```
next()
+
redirect
+
rewrite
+
real use cases
```

And one final correction/clarification from our earlier discussion: **in current Next.js 16, the official terminology is Proxy rather than Middleware**, while older tutorials/codebases may still use `middleware.ts`. Next.js 16 is currently the Active LTS release.  Next.js



Today 5:43 PM

so proxy comes from code to server or server to code?
and also in proxy.ts we are just exporting config how
exactly it knows proxy code to execute just bcoz proxy.ts
file exists??

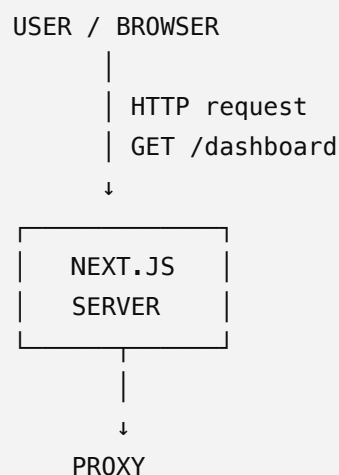
Yes — this is the exact point where the terminology can get confusing. 🤔 Let's separate **browser → server**, **Proxy → route**, and how Next.js discovers `proxy.ts`.

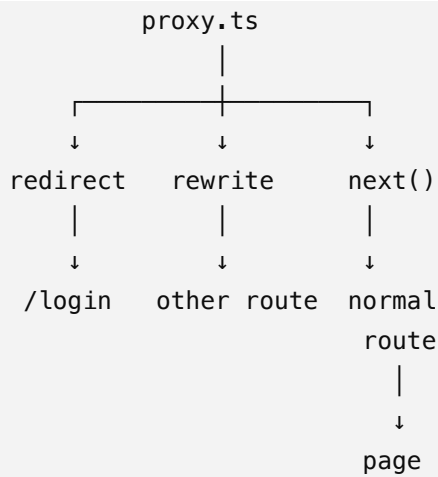
1. Does Proxy happen from code → server or server → code?

Neither, exactly.

Proxy is **server-side code that runs as part of handling an incoming request**.

Think:





So the direction is:

Browser sends request → Next.js server receives it → Next.js executes Proxy → Proxy decides what happens next.

Proxy is **not something the browser calls**.

The browser doesn't do:

```
</> JavaScript
```

```
proxy()
```



It simply sends:

```
</> http
```

```
GET /dashboard
```



and Next.js's server infrastructure invokes your Proxy code.

2. Then how does Next.js know that `proxy.ts` contains the Proxy?

This is the important part.

You don't manually import it.

You don't write:

```
</> TypeScript
```

```
import { proxy } from "../proxy";
```



somewhere.

Instead, **Next.js** has a convention for this special file.

If your project contains:

```
my-app/  
├─ app/  
│   ├─ page.tsx  
│   └─ dashboard/  
│       └─ page.tsx  
├─ proxy.ts  
├─ package.json  
└─ next.config.ts
```



Next.js's build system knows:

"There is a special `proxy.ts` file at the project root. This is the application's Proxy entry point."

It's similar to other Next.js conventions.

For example:

```
app/page.tsx
```



isn't special because you imported it somewhere.

Next.js knows:

```
page.tsx → route UI  
layout.tsx → layout  
loading.tsx → loading UI  
error.tsx → error boundary  
proxy.ts → Proxy entry point
```



These are **framework conventions**.

3. Why do we export `proxy()` ?

Because Next.js needs to know **what code to execute when Proxy runs**.

For example:

</> TypeScript



```
import { NextRequest, NextResponse } from "next/server";

export function proxy(request: NextRequest) {

  console.log("Proxy running!");

  return NextResponse.next();
}
```

Next.js effectively discovers:

proxy.ts



```
↓
find exported proxy function
↓
register it as the request interception function
```

Then an incoming matching request causes:

GET /dashboard



```
↓
Next.js
↓
proxy(request)
↓
your code executes
```

4. What about config?

This part is separate.

You have:

</> TypeScript



```
export function proxy(request: NextRequest) {
  // logic
}

export const config = {
  matcher: ["/dashboard/:path*"],
};
```

There are **two exports** with two different jobs.

proxy

```
</> TypeScript
export function proxy(request) {
  ...
}
```

means:

"Here's the code to execute."

config

```
</> TypeScript
export const config = {
  matcher: [...]
}
```

means:

"Here's when/where this code should run."

So:

```
proxy.ts
├── proxy()
│   └── ↓
│       WHAT TO DO
└── config
    └── ↓
        WHERE/WHEN TO RUN
```

That's the clean mental model.

5. Let's trace an actual request

Suppose you have:

```
</> TypeScript
// proxy.ts
```

```
import { NextRequest, NextResponse } from "next/server";

export function proxy(request: NextRequest) {

  console.log("Proxy executed");

  const token = request.cookies.get("token");

  if (!token) {
    return NextResponse.redirect(
      new URL("/login", request.url)
    );
  }

  return NextResponse.next();
}

export const config = {
  matcher: ["/dashboard/:path*"],
};
```

And your app has:

```
app/
├── login/
│   └── page.tsx
└── dashboard/
    └── page.tsx
```

Now the browser requests:

```
</> http
GET /dashboard
```

Step 1

Request reaches your Next.js server:

```
Browser
  |
  | GET /dashboard
  ↓
Next.js Server
```

Step 2

Next.js checks the Proxy configuration:

```
matcher:  
"/dashboard/:path*"
```



Does `/dashboard` match?

YES 



Step 3

Next.js invokes:

```
</> TypeScript  
proxy(request)
```



Your code runs:

```
</> TypeScript  
const token = request.cookies.get("token");
```



Step 4

No token?

```
</> TypeScript  
return NextResponse.redirect(  
  new URL("/login", request.url)  
);
```



So the request doesn't continue to:

```
app/dashboard/page.tsx
```



Instead:

```
/dashboard  
  ↓  
Proxy  
  ↓  
No token  
  ↓  
Redirect  
  ↓  
/login
```



6. What if the request is `/products` ?

Browser:

```
</> http  
GET /products
```



Next.js sees:

```
matcher = /dashboard/:path*
```



Does:

```
/products
```



match?

```
NO ❌
```



So Proxy isn't invoked for that request.

Then normal routing proceeds:

```
/products  
  ↓  
route resolution  
  ↓  
app/products/page.tsx
```



7. What if we don't have a matcher?

This is where people sometimes get confused.

You can define Proxy without an explicit matcher:

```
</> TypeScript  
  
export function proxy(request: NextRequest) {  
  return NextResponse.next();  
}
```



Then the Proxy can apply broadly according to Next.js's default matching behavior.

That's why `matcher` is useful: it lets you **narrow down the requests that should enter Proxy**.

For example:

`</>` TypeScript

```
export const config = {
  matcher: [
    "/dashboard/:path*",
    "/admin/:path*",
  ],
};
```

Now:

/dashboard	→ Proxy	✓
/dashboard/foo	→ Proxy	✓
/admin	→ Proxy	✓
/admin/users	→ Proxy	✓
/products	→ Proxy	✗
/about	→ Proxy	✗

8. One subtle but important point

The **file itself** is what Next.js discovers because of the framework convention.

But this:

`</>` TypeScript

`proxy.ts`

by itself doesn't mean:

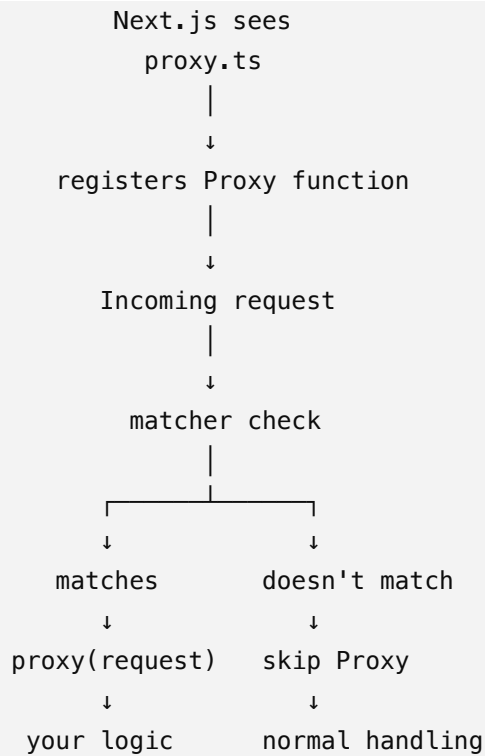
"Run this code for every request."

Next.js needs the expected exported function, and the `matcher/config` determines which requests it applies to.

Think of it like:

Build time





9. Compare this with `page.tsx`

This is actually a great way to understand Next.js conventions.

You don't do:

`</> TypeScript`

```
import DashboardPage from "../app/dashboard/page";
```

to tell Next.js that it's a page.

You create:

`app/dashboard/page.tsx`

and Next.js's router understands:

`dashboard/page.tsx`

↓
`/dashboard`

Similarly:

`proxy.ts`

↓

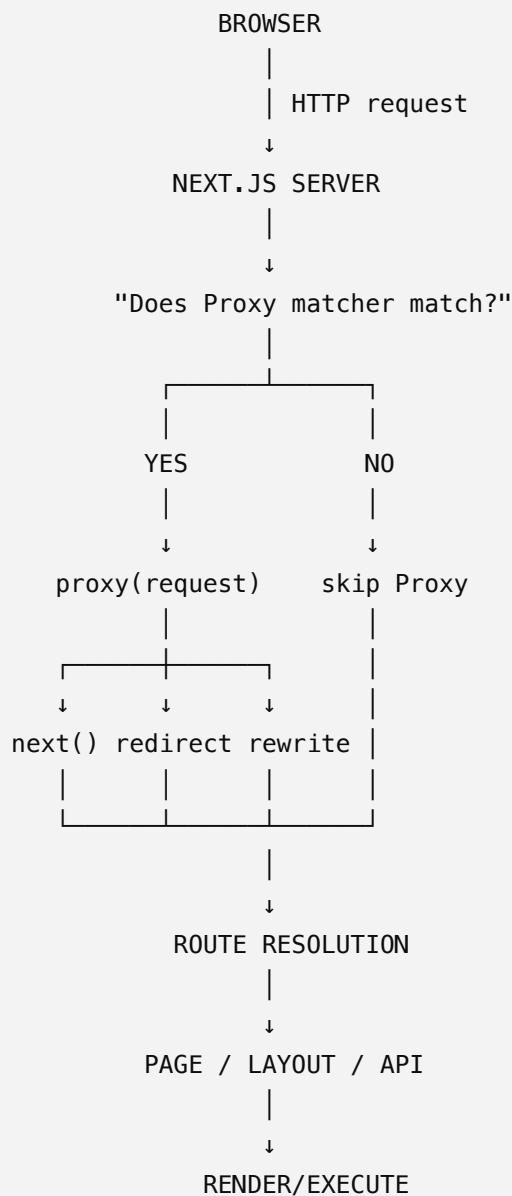
Next.js recognizes special entry point

↓

registers Proxy

This is called **file-system/framework convention**.

10. So remember this exact flow 🧠



The one-sentence interview answer:

"Proxy is server-side request interception. When Next.js builds the application, it recognizes the conventionally named `proxy.ts` entry point and registers its exported Proxy function. For an incoming request, Next.js checks the configured matcher; if it matches, it invokes `proxy(request)`, which can continue, redirect, or rewrite the request before normal route handling proceeds."

And **this** answers your "server → code?" question:

```
HTTP request
  ↓
Next.js server
  ↓
YOUR proxy.ts CODE
  ↓
Next.js continues based on what your code returns
```



So the **server invokes your code**, not the other way around.



Perfect 🤔👍

That means you now have the core picture:

```
Browser
  ↓
HTTP Request
  ↓
Next.js Server
  ↓
Proxy (if matcher matches)
  ↓
redirect / rewrite / next()
  ↓
Route Resolution
  ↓
Page + Layouts
  ↓
Server/Client Rendering
  ↓
RSC Payload + HTML
  ↓
Browser
  ↓
Hydration / Interactivity
```



And the **three Proxy pieces** are:

```
proxy.ts
  ↓
"Here is the Proxy code"

matcher
  ↓
"These requests should enter Proxy"

NextResponse
  ↓
"What should happen to the request?"
  ├── next()
  ├── redirect()
  └── rewrite()
```



You're good to move on from Proxy. 🚀



okay

Awesome 😄

The next natural topic in our roadmap is **Next.js caching + rendering under the hood**, where we'll connect:

Request Memoization → Data Cache → Full Route/Cache Components → Router Cache → revalidation → client navigation

That'll tie together a lot of the questions you've been asking about **"what exactly is stored where?"** and **"what happens on the next request?"**. 🔥



hey we completed these topics

You're right 😄 — **we already completed those topics**. I shouldn't have sent you backward.

Based on what we've covered, you've already done:

- ☒ Nested & dynamic routing
- ☒ Route Groups
- ☒ Parallel Routes
- ☒ Intercepting Routes
- ☒ Server/Client boundaries
- ☒ Serialization
- ☒ RSC Payload / React Flight
- ☒ Hydration
- ☒ Optimistic updates
- ☒ CSR, SSR, SSG, ISR
- ☒ Pre-rendering vs build-time rendering
- ☒ Dynamic/request-time rendering
- ☒ Rendering lifecycle
- ☒ App Router vs Pages Router rendering
- ☒ Request Memoization
- ☒ Route resolution
- ☒ Redirects / permanent redirects
- ☒ Proxy/Middleware
- ☒ Matchers
- ☒ Redirect vs Rewrite

So we should not repeat caching/rendering. 👍

Where we are now

You've essentially finished the big **Next.js architecture + routing + rendering + caching foundation**.

The remaining roadmap should move into the more interview-oriented pieces:

NEXT.JS

- |
- |— ☒ Fundamentals
- |— ☒ App Router & Routing
- |— ☒ Server vs Client Components
- |— ☒ RSC / Flight / Serialization
- |— ☒ Data Fetching
- |— ☒ Rendering
- |— ☒ Caching
- |— ☒ Mutations / Server Actions
- |— ☒ Routing advanced concepts
- |
- |— 🚀 Next major topics
 - |— Middleware / Proxy ☒ just completed
 - |— Error & Loading boundaries
 - |— Metadata & SEO

- |— Image optimization
- |— Font optimization
- |— Script optimization
- |— Route Handlers / APIs
- |— Authentication patterns
- |— Performance optimization
- |— Deployment / production
- |— Next.js interview/system-design scenarios

And since you're ultimately preparing for **frontend interviews**, I'd recommend we now stop expanding the theory endlessly and start doing **scenario-based interview questions + implementation exercises** around these concepts.

For example:

"A user navigates from `/products` to `/products/123` . Explain exactly what happens from click → router → server → RSC payload → browser → hydration."

That's where all the pieces you've learned finally come together. 🔥



ChatGPT can make mistakes. Check important info.